AUTONOMOUS FRAMEWORK FOR REAL-TIME DATA QUALITY, PRIVACY, AND SEMANTIC INTEGRITY IN STREAMING ENVIRONMENTS

Group Report

Project ID: 25-26J-090

A.S. Silva (IT22549136)

L.T.E. Arachchi (IT22893666)

H.A. Amanda K. Arangalla (IT22556684)

W.R.W.M.N.S. Weerakoon (IT22204752)

Dissertation submitted in partial fulfillment of the requirements for the

Bachelor of Science (Hons) in Information Technology

Specializing in Data Science

Department of Information Technology

Sri Lanka Institute of Information Technology

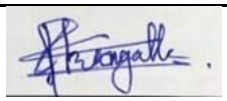
Sri Lanka

August 2025

DECLARATION

We declare that this is our own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute our dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

NAME	STUDENT ID	SIGNATURE	DATE
A. S.Silva	IT22549136		22 / 04 / 2026
L.T.E. Arachchi	IT22893666		22 / 04 / 2026
H. A. Amanda K. Arangalla	IT22556684		22 / 04 / 2026
W.R.W.M.N.S. Weerakoon	IT22204752		22 / 04 / 2026

Certified by:

Name of the Supervisor : Dr. Lakmini Abeywardhana

Signature of Supervisor: _____

Date: 22 / 04 / 2026

Name of the Co-Supervisor : Dr. Mahima Weerasinghe

Signature of Co-Supervisor: _____

Date: 22 / 04 / 2026

DEDICATION

Dedicated to our families, mentors, and the data engineering community whose inspiration and support made this work possible.

ACKNOWLEDGEMENTS

We would like to express our profound gratitude to our dissertation supervisor for the invaluable guidance, continuous encouragement, and expert feedback provided throughout the duration of this research project. The depth of knowledge and constructive criticism offered at every stage significantly shaped the direction and quality of this work.

We extend sincere appreciation to the faculty and staff of the Department of Information Technology at Sri Lanka Institute of Information Technology (SLIIT) for providing the academic infrastructure, resources, and technical environment necessary for carrying out this research.

We are grateful to the data engineering community and the open-source contributors behind Apache Kafka, FastAPI, scikit-learn, HuggingFace Transformers, SHAP, LIME, ChromaDB, MongoDB, and the Groq API, whose frameworks and tools underpinned the practical components of this dissertation.

Special thanks are extended to the developers of the XLM-RoBERTa language model, the Ollama LLM infrastructure, and the LangChain framework, which enabled the intelligent capabilities embedded across multiple components of the proposed system.

Finally, we thank our families for their unwavering moral support, patience, and encouragement throughout the academic journey that culminated in this research.

ABSTRACT

Modern data pipelines face an unprecedented confluence of challenges: heterogeneous data quality defects, privacy exposure of personally identifiable information, structural schema volatility, and the inefficiency of static preprocessing sequences applied uniformly across diverse input modalities. Existing solutions address these concerns in isolation, leaving production streaming environments without an integrated, real-time framework capable of governing data quality, privacy, schema integrity, and preprocessing in a unified manner.

This dissertation presents an Autonomous Framework for Real-Time Data Quality, Privacy, and Semantic Integrity in Streaming Environments a modular, four-component architecture designed for deployment in high-throughput data pipelines. Each component addresses a distinct and critical dimension of streaming data governance.

The first component introduces an Agentic Multi-Modal Unstructured Data Preprocessing Framework with LLM-Guided Orchestration and Quality-Driven Validation. A locally-hosted large language model (LLM) dynamically plans text preprocessing agent sequences, while a rule-based decision engine selects image preprocessing agents based on quantitative quality metrics. Post-transformation validation using PSNR and SSIM thresholds ensures no degradation of already-clean data. The system achieves 88–92% text cleaning success and 92% image transformation acceptance rate with 35–40% mean latency reduction over static pipelines.

The second component presents a Scalable Framework for Structured Data Cleaning leveraging dependency-informed machine learning prediction. Per-column Random Forest models trained exclusively on statistically dependent predictors, identified through chi-square testing, mutual information, and Spearman correlation, produce contextually coherent corrections. A Human-in-the-Loop (HITL) governance layer with SHAP and LIME explainability ensures accountability, while LLM-based rule extraction via LLaMA 3.3-70B automates constraint derivation from domain documents.

The third component delivers an Autonomous Framework for Real-Time PII Detection and Privacy Preservation. A fine-tuned XLM-RoBERTa NER model combined with deterministic regex-based extractors achieves F1 > 0.91 for PII detection. A regulation-aware policy engine spanning GDPR, HIPAA, CCPA, and PDPA drives an adaptive protection module applying AES-256-GCM encryption, tokenization, pseudonymization, differential privacy, and hashing. A Retrieval-Augmented Generation subsystem enables continuous policy evolution from uploaded regulation documents.

The fourth component establishes a Framework for Schema Drift Detection and Governed Adaptation. The framework combines column normalization, multi-feature rename resolution using lexical, semantic, structural, and statistical evidence, dual-mode risk scoring, and a human-in-the-loop governance workflow with approve, reject, and rollback capabilities. The rename model achieves F1 = 0.85 and PR-AUC = 0.9226 on 160 held-out scenarios.

Together, these four components form a cohesive autonomous framework deployable as independent modules or as an integrated pipeline, offering end-to-end streaming data governance across preprocessing, quality, privacy, and schema dimensions.

Keywords: streaming data pipelines, data quality, PII detection, schema drift, agentic AI, LLM orchestration, privacy preservation, Human-in-the-Loop governance, Apache Kafka, GDPR compliance, Named Entity Recognition, Random Forest, SHAP, LIME.

Table of Contents

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AES	Advanced Encryption Standard
BIO	Begin-Inside-Outside (NER tagging scheme)
CCPA	California Consumer Privacy Act
CLAHE	Contrast Limited Adaptive Histogram Equalization
CSV	Comma-Separated Values
DP	Differential Privacy
ETL	Extract, Transform, Load
F1	F1-Score (harmonic mean of precision and recall)
FastAPI	Fast Application Programming Interface (Python web framework)
GDPR	General Data Protection Regulation
GCM	Galois/Counter Mode (AES encryption mode)
HIPAA	Health Insurance Portability and Accountability Act
HITL	Human-in-the-Loop
IoT	Internet of Things
JSON	JavaScript Object Notation
LLM	Large Language Model
LIME	Local Interpretable Model-agnostic Explanations
MAE	Mean Absolute Error
ML	Machine Learning
MPC	Secure Multi-Party Computation
NER	Named Entity Recognition
NIC	National Identity Card
NLMeans	Non-Local Means (denoising algorithm)
NLP	Natural Language Processing
PDPA	Personal Data Protection Act (Sri Lanka No. 9 of 2022)
PHI	Protected Health Information
PII	Personally Identifiable Information
PR-AUC	Precision-Recall Area Under Curve
PSNR	Peak Signal-to-Noise Ratio
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control

Abbreviation	Full Form
REST	Representational State Transfer
RF	Random Forest
SHA	Secure Hash Algorithm
SHAP	SHapley Additive exPlanations
SLIIT	Sri Lanka Institute of Information Technology
SSIM	Structural Similarity Index
SSE	Server-Sent Events
XAI	Explainable Artificial Intelligence
XLM-RoBERTa	Cross-lingual Language Model - Robustly Optimized BERT Pretraining Approach

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The contemporary digital landscape is defined by the pervasive generation and consumption of real-time data streams. Organizations across healthcare, finance, IoT, e-commerce, and enterprise analytics rely on continuously operating data pipelines to power critical decision-making, automated systems, and machine learning workflows. Industry estimates indicate that nearly 80–90% of enterprise data is unstructured, growing at 55–65% compound annual rates, while global data volumes are projected to exceed 181 zettabytes by 2025 [1]. Poor data quality alone costs the United States economy an estimated $3.1 trillion annually, and 95% of businesses cite unstructured data as a major operational concern [2].

Real-time streaming environments introduce a distinct set of challenges that fundamentally differentiate them from batch processing contexts. High-velocity data flows demand sub-100 millisecond latency responses, leaving little room for manual intervention or deferred remediation. Quality defects—including missing values, categorical violations, statistical outliers, and structural inconsistencies—propagate through pipeline stages before detection logic can isolate and quarantine affected records. When such defects reach downstream analytical systems, machine learning models, or production databases, the consequences range from incorrect analytics outputs to regulatory compliance failures and system outages [3].

Four distinct problem dimensions define the technical landscape that this research addresses. First, the preprocessing of heterogeneous unstructured data—spanning text, images, audio, and video—remains dominated by static, hand-crafted pipelines that apply identical operations to every input regardless of whether those operations are warranted. Second, structured tabular data streams suffer from quality degradation through missing values, outliers, and constraint violations that existing automated approaches address without adequate explainability, human oversight, or dependency-awareness. Third, personally identifiable information (PII) within streaming pipelines is typically protected post-ingestion rather than at the ingestion layer, creating a raw exposure window during which sensitive data is vulnerable to insider threats,

misconfigurations, and regulatory non-compliance. Fourth, schema drift—structural changes to upstream data definitions that arrive unannounced—breaks downstream transformation logic, corrupts analytical outputs, and introduces governance exposure that existing detection tools identify but do not manage through governed workflows.

The convergence of these four challenges in a single production pipeline creates a compounded risk environment that no existing unified framework addresses. Individual tools—Apache Airflow, DataGrail, OneTrust, HoloClean, AutoSchema—each address narrow subsets of the problem but share a fundamental architectural limitation: they operate in isolation, addressing individual symptoms while leaving adjacent vulnerabilities unresolved [4][5][6]. This research proposes and evaluates a modular, four-component framework that closes this gap.

1.2 Research Problems

This research addresses four interconnected research problems, each constituting an independent contribution while collectively forming an integrated streaming data governance framework.

Research Problem 1 (Unstructured Data Preprocessing): How can a distributed multi-agent framework autonomously assess and preprocess heterogeneous unstructured data in real time, ensuring scalability, reliability, and end-to-end integrity, with adaptive agent selection that avoids unnecessary processing of already-clean inputs and validates each transformation before commitment?

Research Problem 2 (Structured Data Quality): How can a streaming-compatible data cleaning framework produce contextually coherent corrections for structured tabular data through dependency-informed machine learning prediction, while maintaining human accountability through explainable AI and a governed Human-in-the-Loop review process?

Research Problem 3 (Privacy Preservation): How can raw personally identifiable information be eliminated at the data ingestion layer—before any downstream persistence—through automated multi-method detection and regulation-aware

adaptive protection, ensuring verifiable compliance with GDPR, HIPAA, CCPA, and PDPA without introducing prohibitive processing latency?

Research Problem 4 (Schema Drift Management): How can schema drift in streaming data pipelines be detected, semantically interpreted, risk-scored, and subjected to governed human review with full auditability, rather than being either blocked entirely or absorbed silently, to produce schema adaptation decisions that are simultaneously accurate, operationally safe, and traceable?

1.3 Research Objectives

The overarching objective of this research is to design, implement, and evaluate an autonomous framework for real-time data quality, privacy, and semantic integrity in streaming environments, organized across four component objectives:

- To develop an agentic multi-modal unstructured data preprocessing framework integrating LLM-guided text agent selection and rule-based image agent selection, with per-step PSNR/SSIM quality validation and an Apache Kafka streaming backbone.
- To design and implement a structured data cleaning framework featuring a dependency-informed per-column Random Forest prediction engine, LLM-based validation rule extraction, SHAP and LIME explainability, and a three-mode Human-in-the-Loop governance layer.
- To build an autonomous PII detection and privacy preservation pipeline integrating fine-tuned XLM-RoBERTa NER, deterministic regex extraction, regulation-aware risk scoring, adaptive protection spanning AES-256-GCM, tokenization, hashing, and pseudonymization, and a RAG-based policy enhancement subsystem.
- To construct a schema drift detection and adaptation framework combining column normalization, multi-feature rename resolution, dual-mode risk scoring, and governed human review with approve, reject, and rollback capabilities, supported by a version-controlled schema registry and append-only audit log.

1.4 Significance of the Research

The significance of this research es in its multidimensional nature. From a technical perspective, each of the four components advances the state of the art in its respective domain: the agentic preprocessing framework is the first to combine LLM planning with quantitative quality-metric-based image agent selection and per-step validation reversion within a single Kafka-native architecture; the structured cleaning framework introduces dependency-informed per-column ML prediction as a streaming-compatible alternative to holistic batch repair systems; the privacy framework is the first to achieve zero raw PII at rest through ingestion-layer enforcement rather than post-storage remediation; and the schema drift framework is the first to combine rename-aware structural interpretation, dual-mode risk scoring, and fully auditable governed workflow in a single operational service.

From an applied perspective, the four components address domains with direct regulatory and operational consequences. Healthcare organizations face HIPAA obligations that require PII protection at collection time. Financial institutions operate under GDPR and PCI-DSS mandates that demand data minimization and auditability. IoT-driven platforms contend with high-velocity streams from heterogeneous sensors where schema drift occurs without coordinated change management. The research contributes evidence-based, production-grade solutions to each of these domains.

From an academic perspective, the framework demonstrates that adaptive preprocessing, dependency-informed correction, ingestion-layer privacy enforcement, and governed schema management can coexist as independently deployable modules within a shared streaming infrastructure, enabling organizations to adopt individual components incrementally or deploy the full stack according to governance requirements.

1.5 System Overview

The proposed framework is organized as four independently deployable modules that share a common architectural philosophy: all components operate at or near the data ingestion layer, enforce quality and governance decisions in real time, and maintain complete auditability of every decision made on behalf of the data pipeline.

Component 1 (Unstructured Preprocessing) accepts text strings, delimited files, and image inputs through a FastAPI ingestion endpoint. An Apache Kafka streaming backbone routes each payload to either the agentic text processing pipeline or the rule-based image orchestration pipeline. Text processing uses a locally-hosted Ollama LLM to select preprocessing agents dynamically; image processing uses quantitative quality metrics to gate agent selection. Results are persisted to MongoDB and streamed to a React frontend via Server-Sent Events.

Component 2 (Structured Data Cleaning) receives tabular data batches through a Kafka consumer. A dependency analysis stage constructs a per-column dependency graph before model training. Detected issues—missing values, outliers, and categorical violations are presented through a React-based HITL interface with SHAP and LIME explanations. Human corrections are applied in real time; inactivity triggers a non-destructive auto-fix preview requiring explicit approval.

Component 3 (Privacy Preservation) intercepts data at the ingestion layer through a FastAPI backend. Uploaded files are processed simultaneously by the XLM-RoBERTa NER model and a regex detection engine. Detected PII entities are risk-scored against regulatory policy configurations and protected using the appropriate method before downstream delivery. A RAG subsystem powered by ChromaDB and Google Gemini continuously enriches the policy engine from uploaded regulation documents.

Component 4 (Schema Drift) operates as a plug-in service compatible with existing batch or streaming infrastructure. Each incoming data batch triggers column normalization, schema comparison against the registered canonical schema, rename resolution, and risk scoring. Drift-affected data is isolated to staging while drift-free data proceeds to production. Human reviewers access an approve, reject, or rollback interface; all decisions are recorded in an append-only audit log with full context.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

This chapter reviews the existing body of literature across four research domains central to this dissertation: unstructured data preprocessing with agentic AI orchestration, structured data quality management in streaming environments, privacy preservation in real-time data pipelines, and schema drift detection and adaptation. For each domain, the review covers foundational work, recent advances, and the specific research gap that motivates the corresponding framework component proposed in this research.

2.2 Agentic AI and Unstructured Data Preprocessing

2.2.1 Agentic AI Architectures

The emergence of agentic AI architectures represents a paradigm shift from static scripted pipelines to systems capable of inspecting inputs, reasoning about requirements, and assembling tailored operation sequences. Gioacchini et al. [7] provides a comprehensive taxonomy of agentic AI systems, distinguishing between single-agent and multi-agent systems, tool-augmented planners, and hybrid orchestration approaches. This taxonomy establishes the theoretical grounding for frameworks that deploy a language model as a planner for coordinating specialist processing agents.

The ReAct framework [8] established the foundational pattern of interleaving reasoning traces with action execution in LLM-based agents, demonstrating that language models can plan, act, and adjust their plans based on intermediate observations. HuggingGPT [9] extended this to multi-modal settings by using ChatGPT as a planner routing tasks to specialist models on HuggingFace, demonstrating the viability of planner-plus-specialists' architectures for heterogeneous task routing. However, neither framework was applied to the specific domain of data preprocessing, where the inputs are structured, operations are well-defined, and transformation correctness can be objectively measured.

2.2.2 Text Preprocessing Libraries and Frameworks

Standard NLP preprocessing has historically been handled by libraries including NLTK [10] and spaCy [11]. NLTK's Punkt sentence tokenizer uses an unsupervised algorithm to detect sentence boundaries by learning abbreviation collocations, addressing the non-trivial challenges that naive period-splitting fails to handle. spaCy contributes transformer-backed NER pipelines identifying persons, organizations, locations, and other entity categories across diverse text types. Language identification is handled by langid.py [12], a compact Naive Bayes model over character n-grams achieving strong accuracy across 97 languages with minimal latency. These tools, individually mature and well-validated, have not previously been assembled under dynamic LLM-guided planning that selects them adaptively based on input characteristics.

2.2.3 Image Quality Assessment

Blur detection via Laplacian variance was formally characterized by Pertuz et al. [13], who compared 36 focus measure operators and found the Laplacian-based approach to offer a strong balance of accuracy and computational efficiency. Wang et al. [14] introduced Structural Similarity Index (SSIM) as a perceptually motivated alternative to Peak Signal-to-Noise Ratio (PSNR) for image quality assessment, arguing that mean squared error fails to capture structural information that the human visual system relies on. Contrast enhancement through CLAHE was proposed by Zuiderveld [15]; its tile-based histogram equalization with clip limiting prevents the noise amplification introduced by conventional global histogram equalization. Bilateral filtering [16] and Non-Local Means denoising [17] address different noise regimes, while Canny edge detection [18] enables contour-based region-of-interest extraction.

2.2.4 Streaming Infrastructure for Preprocessing

Apache Kafka [19], originally designed as a distributed messaging system for log processing, has become the industry standard for high-throughput streaming pipelines capable of processing hundreds of thousands of messages per second. Its consumer group model enables horizontal scaling without modifying processing logic, and its fault-isolation properties ensure that failures in one consumer do not propagate to others. MongoDB [20] provides the document storage backend suited to heterogeneous preprocessing result records with nested structure. Server-Sent Events

enable real-time result streaming to frontend monitoring interfaces without the overhead of WebSocket connections.

2.2.5 Research Gap in Unstructured Preprocessing

No prior framework combines LLM-guided agent planning for text preprocessing, quantitative quality-metric-based agent selection for image preprocessing, per-step PSNR/SSIM validation with automatic reversion, and a Kafka streaming backbone within a single production-grade architecture. Existing systems either apply fixed preprocessing sequences regardless of input characteristics or use LLM orchestration without objective validation mechanisms. The framework proposed in this research addresses this gap through an adaptive, measurable, and auditable preprocessing architecture.

2.3 Structured Data Quality in Streaming Environments

2.3.1 Rule-Based Data Cleaning

Rule-based systems employing integrity constraints and denial constraints provide formal consistency guarantees but require domain experts to specify and maintain the full constraint of vocabulary manually. Fan et al. [21] demonstrated that even well-designed constraint sets become incomplete as data sources evolve, creating systematic blind spots. Cong et al. [22] proposed consistency and accuracy frameworks that advance formal guarantees but remain fundamentally dependent on human-authored rule specification. The proposed framework addresses this limitation through LLM-based constraint extraction from unstructured domain documents.

2.3.2 Statistical and ML-Based Repair

Matrix factorization imputation [23] and deep generative approaches, including GAIN [24] achieve strong batch-mode correction accuracy. HoloClean [25] advances the state of the art by combining integrity constraints with probabilistic graphical models for joint repair. However, all three approaches share a critical limitation for streaming deployment: computational complexity that scales poorly with dataset dimensionality and batch arrival rate. The proposed framework achieves streaming-compatible latency by training lightweight Random Forest models per column rather than optimizing a global probabilistic model.

2.3.3 Dependency-Aware Correction and XAI

The importance of inter-column dependencies for correction quality is established in the data cleaning literature [25][26]. HoloClean incorporates dependencies implicitly through its integrity constraint formulation; ActiveClean [27] uses active learning to direct human labeling effort toward dependency-critical attributes. SHAP [28] (SHapley Additive exPlanations) provides global feature importance through exact Shapley value computation for tree models. LIME [29] (Local Interpretable Model-agnostic Explanations) provides locally faithful linear approximations around specific prediction instances, covering cases where SHAP global attributions may be misleading due to feature interaction effects. Combining both explanation techniques provides complementary coverage that enables genuine human oversight rather than the appearance of oversight.

2.3.4 Human-in-the-Loop Data Management

HITL approaches have been applied in data integration, entity resolution, and schema governance [27][30]. Bontempelli et al. [30] established that human review is necessary when automated systems face ambiguous cases that cannot be safely resolved from data alone—a finding directly applicable to ML-based data correction in regulated domains. The proposed framework applies HITL at the individual correction decision level, with every fix decision traceable to either a named reviewer or an auto-apply fallback requiring explicit approval confirmation. This distinguishes the approach from ActiveClean, which incorporates HITL into model retraining but not correction-level human accountability.

2.3.5 LLMs in Data Preparation

Table-GPT [31] demonstrated that fine-tuned language models can perform diverse table-understanding tasks including type inference and value imputation. Entity matching using LLMs [32] achieves state-of-the-art performance through semantic understanding unavailable to string-matching approaches. LLaMA-based models accessed through the Groq API have demonstrated competitive performance on structured information extraction tasks. The proposed framework extends LLM application to validation rule extraction, combining automated constraint generation with an operational streaming cleaning pipeline.

2.3.6 Research Gap in Structured Data Cleaning

No existing framework simultaneously provides dependency-informed ML prediction, automated LLM rule extraction, per-correction XAI explanation, and HITL governance with inactivity fallback within a production-grade streaming architecture. The central gap is the absence of dependency awareness: prior systems either apply corrections column-by-column without accounting for inter-attribute relationships or use globally trained models that conflate signal from unrelated columns with genuine predictive structure. The proposed framework's dependency graph construction addresses this gap explicitly.

2.4 Privacy Preservation in Streaming Data Pipelines

2.4.1 Privacy-Preserving Techniques

A wide range of cryptographic, statistical, and governance-based techniques have been proposed for privacy preservation in data pipelines. Mittoor et al. [33] highlight the challenge of balancing scalability, compliance, and performance in modern data architectures, noting that most privacy controls remain post-ingestion. Mahendra and Verma [34] provide a comprehensive review of scalable privacy-preserving approaches for AI pipelines, covering Differential Privacy (DP), Homomorphic Encryption (HE), Secure Multi-Party Computation (MPC), and Federated Learning (FL). Differential Privacy adds calibrated noise to outputs to prevent individual re-identification while preserving statistical utility [34][35]. Homomorphic Encryption enables computation on encrypted data without decryption but introduces prohibitive computational overhead for real-time systems [34]. Federated Learning retains raw data at source systems and exchanges only model updates, aligning with GDPR data minimization principles [33][35].

Governance-oriented approaches such as tokenization, pseudonymization, and role-based access control complement cryptographic techniques. Kantarcioglu and Shaon [36] introduce SECUREDL, a broker-layer framework that intercepts user access requests and enforces policy, auditing, and on-the-fly sanitization before data reaches underlying databases. AES-256-GCM provides authenticated symmetric encryption with nonce-based integrity protection, making it the standard for high-throughput

encryption where reversibility is required and where authenticated decryption prevents undetected tampering [37].

2.4.2 NER-Based PII Detection

Named Entity Recognition has emerged as the primary mechanism for automated PII identification in unstructured text. Kotadia et al. [38] demonstrated that spaCy-based NER achieves F-measures above 0.9 for de-identification across medical IME reports. Hybrid approaches combining rule-based NLP with transformer NER have shown precision of 94.7% and F1-score of 91.1% on financial PII datasets [39]. However, Agarwal et al. [40] caution that NER-based masking models exhibit critical limitations in handling ambiguous, polysemic, and domain-specific content, motivating ensemble detection strategies. XLM-RoBERTa [41], a cross-lingual language model pre-trained on 100 languages, provides the multilingual NER foundation that the proposed framework fine-tunes for PII entity types, enabling detection across multilingual and domain-specific text streams.

Microsoft Presidio [42] is a prominent open-source framework for PII detection and anonymization, offering context-aware recognizers using NER, regular expressions, and rule-based logic across multiple languages. The proposed framework differs from Presidio in its regulation-aware risk scoring, RAG-based dynamic policy enhancement, and ingestion-layer enforcement that eliminates raw PII before any downstream persistence rather than masking on request.

2.4.3 Real-Time Streaming Privacy

Raptis and Passarella [43] systematically survey Apache Kafka across algorithms, networks, data, and security domains, establishing it as the leading solution for networked data streaming. Zhao et al. [44] propose PrivStream, a privacy-preserving IoT streaming inference framework on Kafka combining deep-learning feature filtering with differential privacy noise injection at the edge. Kodakandla [45] examines privacy-centric data pipeline architectures for generative AI workloads. Despite these advances, a persistent gap remains: most frameworks apply protections after ingestion, leaving a raw PII exposure window. Commercial tools including Protecto AI, Apache Airflow, Dagster, DataGrail, and OneTrust each address specific

aspects of privacy governance but uniformly assume the unavoidable presence of raw PII in storage [46].

2.4.4 Regulatory Compliance Frameworks

The EU General Data Protection Regulation (GDPR, 2018) mandates data minimization, purpose limitation, and privacy by design across all personal data processing operations [47]. The Health Insurance Portability and Accountability Act (HIPAA) establishes specific requirements for Protected Health Information (PHI) in US healthcare contexts [48]. The California Consumer Privacy Act (CCPA) and its successor CPRA grant consumers rights over personal data collection and processing [49]. Sri Lanka's Personal Data Protection Act No. 9 of 2022 (PDPA) establishes the first comprehensive data protection regime in Sri Lanka, imposing obligations on data controllers and processors handling personal data of Sri Lankan residents [50]. The proposed framework's policy engine encodes requirements from all four regulatory frameworks, enabling multi-jurisdictional compliance from a single deployed instance.

2.4.5 Research Gap in Privacy Preservation

The fundamental gap identified across the reviewed literature is the post-ingestion timing of existing privacy enforcement. No existing framework eliminates raw PII at the ingestion layer before any downstream persistence. All reviewed commercial and open-source tools either assume raw PII will exist in storage or apply masking on request rather than at ingestion time. The proposed framework closes this gap through ingestion-layer enforcement, making it the only solution that achieves zero raw PII at rest as a design invariant rather than a governance policy goal.

2.5 Schema Drift Detection and Adaptation

2.5.1 Schema Evolution Theory

Chillón et al. [51] built the Orion language on top of a unified schema model spanning NoSQL and relational databases, formalizing a taxonomy of schema change operations verified with the Alloy model checker. This taxonomy directly informs the severity ordering in the proposed risk engine. The same group extended this work by introducing relational concept stability, tracking whether a schema element retains its

identity across repeated comparisons—a longitudinal perspective that the schema registry is designed to support [52]. Rahm and Bernstein [53] established the foundational taxonomy for automatic schema matching across schema-level, instance-level, element-level, and structure-level dimensions, a classification that maps directly to the rename detection design.

2.5.2 Schema Drift Detection in Practice

Chintakindhi [54] built an AI-based detection pipeline for schema drift in cloud migrations, reporting a 30% reduction in compliance errors. Detection is treated as the terminal step, leaving lifecycle management—risk assessment, governed response, auditable rollback—unaddressed. Vaddepalli [55] introduced AutoSchema, achieving 98.3% detection accuracy with sub-second adaptation. AutoSchema is fully autonomous, efficient but unsuitable for regulated pipelines where every schema change requires an explicit decision record. Sirimalla [56] validated the schema registry and version-comparison architecture that the proposed framework uses. Chitnis and Tewari [57] found that hybrid statistical and ML detection substantially outperforms rule-based thresholds.

2.5.3 Schema Matching and Rename Detection

Asif-Ur-Rahman et al. [58] showed that combining schema-level features with instance-level statistical similarity outperforms either alone, informing the multi-feature rename detection design. Priya and Thilagam [59] demonstrated that field semantics encoded as dense embeddings enable better schema grouping than string similarity, motivating the semantic evidence component. Sheetrit et al. [60] introduced ReMatch, achieving state-of-the-art matching using retrieval and large language models a natural upgrade path for the rename component identified in future work.

2.5.4 Explainability and Governance

Lee et al. [61] demonstrated the value of attaching SHAP-based explanations to drift alerts, motivating the structured explanation payloads in the proposed framework's

governance interface. Abraham et al. [62] define data governance in terms of authority, accountability, and decision mechanisms over data risk, precisely what the approve, reject, and rollback workflow and audit log implement. Bontempelli et al. [30] showed that human-in-the-loop handling is necessary when automated systems face ambiguous drift that cannot be safely resolved from data alone—the rationale for the unconditional staging gate applied to all detected drift events.

2.5.5 Research Gap in Schema Drift Management

Although prior work has addressed schema evolution, schema matching, drift detection, and governance, these areas are usually treated separately. Existing approaches either stop at detecting drift, treat renames as ordinary additions and removals, or lack a governed response process that accounts for operational risk and human review. This leaves a clear gap for a runtime framework that not only detects schema drift, but also resolves ambiguous changes, scores their risk, and supports auditable decision-making before production promotion.

CHAPTER 3: METHODOLOGY

3.1 Introduction

This chapter describes the design, architecture, and implementation of each of the four framework components. The presentation follows a consistent structure for each component: overall architecture, key technical mechanisms, technology stack, and implementation details. The chapter emphasizes the novel contributions of each component relative to the existing literature reviewed in Chapter 2.

3.2 Component 1: Agentic Multi-Modal Unstructured Data Preprocessing

3.2.1 System Architecture

The framework is organized into four functional layers: a REST and Server-Sent Events API layer built with FastAPI, a Kafka-based message streaming layer, a processing layer containing separate text and image orchestrators, and a persistence layer backed by MongoDB. Inputs arrive at the /ingest endpoint as plain text strings, delimited text files (CSV, JSONL, TXT), or image files. Text files are immediately split row-by-row before being enqueued, which keeps individual Kafka messages small and independent regardless of source file size. Images are written to disk and a path reference is enqueued instead of raw bytes, avoiding Kafka message size limits.

A router worker subscribes to the raw_input topic, inspects each payload's type signature, and forwards the message to either the detected_text or detected_image topic. Separate Kafka consumer threads subscribe to each routed topic. The text worker passes payloads to the AgenticTextProcessor, which internally handles LLM planning and agent execution. The image worker loads the image from disk, converts it to an RGB NumPy array, and passes it to the ImagePreprocOrchestrator. A final results worker subscribes to both processed topics, normalizes the record structure, upserts the document into MongoDB, and appends it to an in-memory SSE buffer consumed by the React frontend.

Five Kafka topics structure the pipeline: raw_input (ingestion endpoint to router), detected_text (router to text worker), detected_image (router to image worker), processed_text (text processor to results worker), and processed_image (image

orchestrator to results worker). This topic architecture ensures fault isolation between text and image processing while enabling independent horizontal scaling of each worker type.

3.2.2 LLM-Guided Agent Planning for Text Preprocessing

The AgenticTextProcessor delegates agent selection to an OllamaPlanner instance, which wraps a locally-hosted LLM via the LangChain Ollama interface. For each incoming text sample, the planner constructs a prompt that presents the input alongside a fixed menu of eight available agents and instructs the model to return a JSON array of agent identifiers. The eight available agents are: html_strip_agent, lang_detect_agent, translator_agent, spell_agent, sentence_split_agent, tokenizer_agent, ner_agent, and validation_agent.

The model's response is parsed and reordered into a canonical execution sequence to preserve logical dependencies: HTML stripping must precede language detection because markup can confuse the classifier; language detection must precede translation; spell correction must apply to English text only and therefore must follow the translator. The canonical ordering enforces these dependencies regardless of the order in which the LLM lists agents. If the LLM is unavailable or returns malformed JSON, a conservative fallback plan activates three agents: spell_agent, tokenizer_agent, and lang_detect_agent.

One deliberate design constraint is that the LLM is never permitted to generate or modify text. Its role is strictly classificatory: given the input, output a list of agent names. All actual transformation logic resides in deterministic, auditable Python functions. This separation ensures that the system remains functional if the LLM is replaced, upgraded, or temporarily unavailable, because the fallback plan covers the most common preprocessing needs without any LLM involvement.

3.2.3 Text Processing Agents

Each text agent is designed as a stateless, self-evaluating unit exposing three properties: a confidence score in [0, 1], a boolean should_run flag encoding preconditions, and a changed flag indicating whether the agent modified its input. The HTML Strip Agent applies a two-pass regex removing all HTML tags before

collapsing residual whitespace. Language detection uses langid.py classifying text against 97 languages using character n-gram features. The Spell Correction Agent computes a noise ratio as the count of out-of-dictionary tokens divided by total token count; if this ratio falls below 0.10, the agent returns the text unchanged. The Sentence Splitter and Tokenizer agents wrap NLTK's sent_tokenize and word_tokenize respectively. The NER Agent passes cleaned English text to spaCy's en_core_web_sm pipeline, extracting PERSON, ORG, GPE, PRODUCT, DATE, TIME, LAW, and EVENT entities. The Validation Agent uses Pydantic to schema-validate the assembled output record against a typed CleanedTextRecord model.

The Translator Agent introduces a dynamic plan modification at runtime: when the Language Detection Agent identifies non-English input, the execution engine inserts the Translator Agent into the remaining plan immediately after the current step, before any English-specific agents execute. This insertion is a rule applied by the execution engine, not a re-query to the LLM, reflecting the hard dependency between language and downstream agent eligibility.

3.2.4 Rule-Based Decision Engine for Image Preprocessing

The RuleBasedDecisionEngine computes three scalar quality metrics from each input image before selecting any agents. Blur is quantified as the variance of the Laplacian response, validated empirically by Pertuz et al. [13] as one of the most reliable no-reference focus operators. The Laplacian kernel is convolved with the grayscale image, and the variance of the resulting response map is computed. Sharp images produce high variance; blurred images yield low variance.

Brightness is the mean luminance of the grayscale image, computed using the ITU-R BT.601 weighted conversion: Gray(i,j) = 0.299R + 0.587G + 0.114B. Contrast is the standard deviation of the grayscale pixel distribution. Given the three metrics, the decision engine applies threshold logic: the ROI agent is unconditionally included; the deblur agent is added when blur falls below 120.0; the enhancement agent is added when brightness falls outside [70.0, 200.0]; both enhancement and denoise are added when contrast falls below 60.0. The total action list is capped at three agents to prevent cascading transformations on marginal data.

3.2.5 Image Processing Agents and Post-Transformation Validation

The ROI Agent converts the image to grayscale, applies Gaussian blur to suppress noise before edge detection, runs Canny edge detection with thresholds of 50 and 150, identifies external contours, and crops to the bounding rectangle of the largest contour if its area exceeds 5% of the total image area. The Enhancement Agent applies gamma correction followed by CLAHE on the L channel of the LAB color space, using a tile size of 8 and clip limit of 2.0. The Denoise Agent defaults to Non-Local Means denoising, exploiting patch-level self-similarity for noise reduction while preserving fine texture. The Deblur Agent implements Wiener deconvolution in the frequency domain: F_hat(u,v) = H*(u,v) / (|H(u,v)|^2 + lambda) * G(u,v), where lambda = 0.01 is the regularization parameter.

After every agent execution, the QualityValidator compares before and after images using PSNR and SSIM. A transformation is accepted only if SSIM >= 0.995 and PSNR >= 15.0 dB. If either condition fails, the agent output is discarded and the pipeline continues from the pre-agent state. This revert mechanism prevents cascading quality loss from poorly-configured transformation passes.

3.2.6 Technology Stack

The technology stack comprises: FastAPI (Python) for the REST/SSE API layer; Apache Kafka for message streaming with five topics; Ollama with LangChain for locally-hosted LLM inference; NLTK and spaCy for text processing; langid.py for language detection; pyspellchecker for spell correction; OpenCV and NumPy for image processing; MongoDB for result persistence; and React 19 with Server-Sent Events for the monitoring frontend.

3.3 Component 2: Structured Data Cleaning Framework

3.3.1 System Architecture

The framework adopts a layered microservices architecture, decomposing the cleaning pipeline into independently scalable components across five functional layers: user interaction, API orchestration, Kafka streaming, ML/LLM processing, and persistent storage. Table 3.1 summarizes the component technologies and their assigned responsibilities.

TABLE 3.1. System Component Summary

Component	Technology	Responsibility
Frontend	React + Vite	HITL review UI; inactivity timer; approve/rollback modal
Backend API	FastAPI (Python)	Session mgmt; pipeline orchestration; fix preview & commit
Streaming	Kafka + Zookeeper	Fault-tolerant batch ingestion; partition-level scalability
ML Engine	Scikit-learn RF	Per-column dependency-trained classifiers and regressors
XAI Layer	SHAP + LIME	Per-fix attributions surfaced in the HITL review interface
LLM Module	Groq / LLaMA 3.3-70B	Automated validation rule extraction from documents
Storage	Session store + CSV	In-memory state; audit trail; cleaned dataset export

The React/Vite frontend hosts the HITL review interface. It presents each detected issue alongside its XAI explanation and available fix options, implements inactivity detection through pointer, keyboard, mouse, and touch event tracking, and renders the approval modal for inactivity-triggered auto-fix previews. The FastAPI backend exposes two correction endpoints: POST /fixes/{session_id} for committing corrections and POST /fixes/{session_id}/preview for non-destructive preview generation.

A distributed streaming pipeline is implemented using the Confluent Platform distributions of Apache Kafka and Zookeeper. Data arrives in configurable batches (default: 500 records per batch) via either the Kafka consumer or direct file upload. Fault tolerance is achieved through atomic file writes and persistent batch storage in the consumed_batches directory, enabling seamless recovery from system failures without data loss. The architecture supports horizontal scaling through increased partition counts and consumer group instances, with Kafka-based decoupling providing backpressure management to absorb transient throughput spikes.

3.3.2 LLM-Based Rule Extraction

The LLaMA 3.3-70B model, accessed via the Groq API, automatically extracts validation rules from unstructured domain documents including PDFs and plain-text files. A structured prompt instructs the model to identify and serialize field-level constraints into a machine-readable CSV format with six columns: Column, dtype,

min, max, allowed_values, and constraints. Post-processing normalizes and validates extracted rules before integration. The system supports iterative expert refinement, allowing domain specialists to review and augment automatically derived constraints without system redeployment. Supported dtype values include integer, float, categorical, text, and boolean. This approach eliminates the manual constraint specification requirement that limits rule-based systems to well-documented, stable domains.

3.3.3 Dependency Analysis and Column Classification

Before any model is trained, the system constructs a dependency graph over the dataset columns. Identifier columns are excluded using three heuristics: the column name contains the substring "id"; the uniqueness ratio exceeds 0.95; or the column contains near-integer values with a sequential increment ratio above 0.90. For remaining columns, pairwise statistical dependencies are computed using three complementary measures: chi-square test of independence for categorical-categorical pairs (significance threshold $p < 0.05$); mutual information regression for mixed categorical-numerical pairs (normalized to [0, 1]); and Spearman rank correlation for ordinal and continuous numerical pairs (threshold $|r| > 0.30$). Only column pairs exceeding the respective significance threshold are retained in the dependency graph, ensuring that each model receives only genuinely predictive features.

3.3.4 Dependency-Informed ML Prediction Engine

The primary technical contribution of this framework is the dependency-informed ML prediction engine. For each non-identifier column appearing as an issue target, a dedicated supervised learning model is trained using only the column's dependency-validated predictors as features. Two model types are instantiated based on the target column's inferred data type: Random Forest Classifier (n_estimators=50, max_depth=10) for categorical columns, and Random Forest Regressor (n_estimators=50, max_depth=10) for numeric columns. Preprocessing uses a scikit-learn Pipeline with a ColumnTransformer: numerical predictors pass through a SimpleImputer (strategy=median); categorical predictors pass through a SimpleImputer (strategy=most_frequent) followed by OneHotEncoder.

For each detected issue, the prediction engine generates a structured fix record containing: the ML suggestion (primary prediction); a cap-to-boundary suggestion for outliers; a ranked list of statistical alternatives (mean, median, mode, IQR quartiles); and an XAI explanation from SHAP and LIME. The auto-selection logic, used only in fallback modes, iterates over fix candidates in priority order and selects the first candidate that passes a local validation check confirming resolution of the detected issue without violating any extracted rule or IQR constraint.

3.3.5 SHAP and LIME Explainability

Each fix suggestion is accompanied by an explainability payload generated using two complementary techniques. SHAP TreeExplainer computes exact Shapley values identifying the magnitude and direction of each predictor's contribution to the suggested fix value. LIME provides locally faithful linear approximations of the model's decision boundary around each specific issue instance, covering cases where SHAP global attributions may be misleading due to feature interaction effects. Both explanations are rendered in the HITL review interface, enabling informed reviewer decisions rather than blind suggestion acceptance. A reviewer who sees that a salary imputation is driven primarily by department and seniority can confirm or reject the suggestion in seconds rather than independently verifying from domain knowledge.

3.3.6 HITL Correction Governance

The HITL governance layer structures correction decisions across three operational modes. Mode 1 (Active Review): the user reviews each issue, examines the ML suggestion with its XAI explanation, and explicitly selects one option. No correction is applied until the user submits the full review, providing complete reviewer accountability. Mode 2 (Partial Review with Auto-Fill): user selections are applied exactly; unreviewed issues receive auto-selected fixes following priority ordering, clearly distinguished in the session audit trail. Mode 3 (Inactivity-Triggered Preview): when the user has not interacted with the review interface for the configured inactivity duration (default: 60 seconds), the system generates a non-destructive preview server-side. The approval modal displays quality scores before and after proposed corrections. The user must explicitly choose Approve or Rollback before any change is committed. Four safeguards ensure operational safety: inactivity detection tracks all user input

channels; the preview endpoint never modifies the session dataframe; approval requires explicit user action with no auto-approval timeout; and rollback suppresses the timer to prevent repeated triggering.

3.3.7 Data Quality Scoring

A composite scoring mechanism quantifies dataset reliability before and after correction. The score is a weighted sum of three dimensions: Completeness (40%, based on missing value count relative to total cells), Validity (40%, based on outlier and invalid entry count relative to total cells), and Consistency (20%, based on duplicate row count and low-variance column detection). The raw weighted sum is normalized to a 0–95 range to account for irreducible noise inherent in high-velocity streaming data, reflecting the epistemic position that perfect quality is unattainable in production streaming environments. The normalized score maps to a five-level grade scale: A (90+), B (75–89), C (60–74), D (45–59), F (below 45).

3.4 Component 3: PII Detection and Privacy Preservation

3.4.1 System Architecture

The proposed framework is organized into five functional layers operating in sequence at the data ingestion point: Data Ingestion and Buffering, Multi-Method PII Detection, Policy Engine and Risk Assessment, Adaptive Protection Module, and Downstream Delivery. The platform employs a modern, modular technology stack: a React 19 + Vite frontend for user interaction; a FastAPI (Python) async backend for API orchestration; PyTorch and HuggingFace Transformers for NER inference; ChromaDB for vector storage of regulation documents; the Python Cryptography library for AES-256-GCM encryption; and Google Gemini API for RAG-based policy generation.

User-uploaded files (TXT, JSON, CSV) are transmitted to the FastAPI backend, where text extraction is performed using format-specific parsers. The extracted text is simultaneously processed by the NER model and the regex detection engine. Results are merged via a priority-based conflict resolution algorithm before being forwarded to the policy engine. Risk scores drive method selection in the protection module, and transformed content is returned to the user or committed to protected storage.

3.4.2 Multi-Method PII Detection

The detection engine employs two complementary strategies to maximize recall and precision across structured and unstructured data. The NER component uses a locally hosted XLM-RoBERTa model fine-tuned for multilingual PII detection with BIO (Begin-Inside-Outside) tagging. Supported entity types include PERSON (PER), LOCATION (LOC), ORGANIZATION (ORG), EMAIL, MARITAL_STATUS, PASSWORD, and SEXUAL_ORIENTATION. A confidence threshold of 0.85 filters low-confidence predictions. The model runs on GPU where available via PyTorch CUDA support.

The deterministic regex engine provides high-confidence, rule-based extraction for structured PII types specific to the Sri Lankan context: National Identity Card (NIC) numbers validated against old (9-digit + V/X) and new (12-digit) formats including day-of-year validation; credit card numbers verified via the Luhn algorithm and issuer prefix matching (Visa, MasterCard, Amex, Discover); bank account numbers detected using known Sri Lankan banking prefixes (Sampath: 01, People's Bank: 2042); and mobile/landline phone numbers normalized to the 0XXXXXXXXX format from +94 equivalents. IPv4, IPv6, and MAC addresses are also extracted with full format validation.

3.4.3 Entity Merging and Policy Engine

Detection outputs from both subsystems are merged via a priority-based algorithm. Regex detections are assigned priority 100 and override NER predictions for overlapping spans. Where multiple detections share the same (entity type, normalized value) tuple, the highest-confidence instance is retained. The policy engine is driven by a machine-readable JSON configuration encoding 15+ entity types with their regulatory citations, sensitivity levels, and protection rationale. Supported frameworks include GDPR, HIPAA, CCPA, PCI-DSS v4.0, and PDPA No. 9/2022.

Each detected PII entity is assigned a composite risk score: RS = 0.7 * S + 0.3 * C, where S is the sensitivity weight (LOW=1, MEDIUM=3, HIGH=5, CRITICAL=5) and C is the context weight reflecting data usage context (logging=1, storage=2,

analytics=3, external_transfer=5). This formula ensures that the same entity type receives escalating protection as data moves toward external exposure.

3.4.4 Adaptive Protection Module

Protection methods are selected dynamically based on RS thresholds: RS ≤ 2.0 applies MASKING (partial obscuration preserving format, e.g., XXXX-XXXX-XXXX-1234 for credit cards); 2.0 < RS ≤ 3.5 applies SHA-256 Hashing (salted one-way hash for non-reversible protection); 3.5 < RS ≤ 4.5 applies AES-256-GCM Encryption (authenticated symmetric encryption with Base64-encoded nonce + ciphertext output, reversible with key); RS > 4.5 applies TOKENIZATION (vault-based replacement with structurally consistent tokens, supports detokenization); PASSWORD entities always receive BCRYPT (adaptive-cost hashing with verification capability); low-risk DATE/LOC entities receive GENERALIZATION (date reduced to year, city reduced to country).

Protection is applied in a longest-first entity ordering to prevent span conflicts during in-place string substitution. All transformations are logged with method identifier, timestamp, and regulatory citation to produce a full audit trail.

3.4.5 RAG-Based Policy Enhancement

A Retrieval-Augmented Generation subsystem enables continuous policy enrichment from regulatory documents. Uploaded PDF regulation texts are processed through a four-stage pipeline: text extraction via PyPDF2; semantic chunking using SpaCy sentence segmentation with 500-character windows and 100-character overlap; embedding via SentenceTransformer ("all-MiniLM-L6-v2"); and storage in a ChromaDB persistent vector database. At query time, the top-5 semantically relevant regulation chunks are retrieved and provided as context to Google Gemini, which generates structured JSON policy recommendations. These are validated and appended to the policy_engine.json, enabling the framework to evolve with new regulatory requirements without code modifications.

3.5 Component 4: Schema Drift Detection and Governed Adaptation

3.5.1 Framework Architecture

The framework processes each incoming batch through four tightly connected stages: schema drift detection, rename resolution, risk scoring, and governance-based routing. Two inputs are provided at dataset registration time: the canonical schema (the approved structural reference) and an optional baseline profile derived from representative historical data that substantially improves rename resolution accuracy. The framework is delivered as a plug-in service that integrates into existing batch or streaming infrastructure without requiring changes to the underlying pipeline.

3.5.2 Column Normalization and Schema Construction

A significant share of apparent schema drift originates from naming convention inconsistencies rather than semantic change. The same logical field can arrive as Customer ID, customer_id, or customerID from different source systems. Before any structural comparison, the framework applies a deterministic normalization function: convert the name to lowercase; replace all non-alphanumeric characters with underscores; collapse consecutive underscores into a single underscore; strip leading and trailing underscores. After this transformation, Customer ID, customer_id, and customer-id all produce customer_id, eliminating an entire class of spurious drift events.

The observed schema is constructed directly from incoming batch data rather than accepting declarations from the source system, which are frequently stale or incomplete. For each column, the system records three properties: normalized name, inferred data type, and a nullability flag. Type inference follows a strict precedence order: Boolean, Integer, Float, Numeric string, Datetime string, String. Conservative inference avoids over-claiming: if any ambiguity exists, the system falls back to the broader type.

3.5.3 Structural Change Identification

The observed schema is compared against the canonical schema to produce a raw structural diff, following the schema change taxonomy of Chillón et al. [51]. Four direct change categories are identified: Additions (columns present in observed but absent from canonical, extending the contract without breaking existing consumers); Removals (columns expected in canonical but missing from observed, the highest-risk

change as downstream consumers dependent on removed fields receive null or raise key errors); Type changes (columns present in both schemas with incompatible inferred types, which silently break arithmetic operations and type-dependent transformations); and Nullability changes (columns where canonical treats the field as non-nullable but the observed batch contains null values, flagged only in this risky direction).

3.5.4 Multi-Feature Rename Resolution

Without rename detection, every column rename produces two entries in the raw diff: one removal and one addition. The removal is assigned the highest severity in the risk model, causing rename events to be scored as highly dangerous when they are often routine refactoring. For each candidate pair formed by a removed canonical column and an added observed column, the system constructs an eight-feature vector drawing on four evidence types:

- Lexical evidence: character-level name similarity using edit distance, token overlap counting shared words after splitting on underscores and camelCase boundaries, and token Jaccard similarity. These features capture obvious renames such as customer_id → cust_id.
- Semantic evidence: each column is described in natural language from its name, type, and example values. That description is encoded with a sentence embedding model and compared using cosine similarity. This captures semantic renames where the underlying concept is the same but vocabulary differs entirely—such as transaction_amount → payment_value.
- Structural evidence: type compatibility between the two candidate columns provides supporting evidence; an exact type match is strong evidence, a numeric-to-numeric match provides partial credit, and incompatible types reduce the rename score.
- Statistical evidence: when a baseline profile is available, the system compares null-rate difference, mean difference, standard deviation difference, and range overlap. Two columns representing the same field under different names will have statistically similar distributions.

The eight features are passed to a trained probabilistic classifier returning a score in [0, 1] representing the likelihood of a genuine rename. Matching is constrained to be one-to-one: a single observed column cannot be assigned to multiple canonical columns. Candidate pairs exceeding the acceptance threshold (0.5359) are accepted as renames and removed from the raw addition and removal lists.

3.5.5 Dual-Mode Risk Scoring

Both scoring modes apply a shared severity ordering: column deletion (severity 1.0), type change (0.9), rename (0.7), nullability change (0.5), addition (0.4). This monotonically decreasing scale reflects blast radius: the wider the downstream impact of a change reaching production undetected, the higher its assigned severity.

Option A (Dataset-Level Scoring) computes: risk_A = clip(0.55T + 0.20C + 0.20S + 0.05K + 0.10U, 0, 1), where T is technical impact (min(1, s_max + 0.08 * ln(1 + n))), C is dataset criticality, S is sensitivity class, K is the key-field flag, and U is rename uncertainty. For regulated datasets, policy-based risk floors override the formula: deletions and type changes on strictly regulated datasets are floored at 0.85 regardless of formula output. Routing thresholds: score >= 0.70 triggers high risk staging; 0.35–0.70 triggers medium risk staging; below 0.35 is low risk.

Option B (Field-Level Scoring) scores each detected change individually: raw_i = (severity_i * I_i * M_i * R) + P_tol,i + P_conf,i, where I_i is impact combining dataset and field criticality, M_i is a semantic-role multiplier (Identifier: 1.0, Foreign key: 0.9, Timestamp: 0.8, Measure: 0.7, Dimension: 0.6), R is a sensitivity multiplier (non-sensitive: 1.0, Internal: 1.05, PII: 1.15, lightly regulated: 1.25, strictly regulated: 1.35), P_tol,i is a tolerance penalty for policy violations, and P_conf,i is a confidence penalty for uncertain renames. Individual change risks combine using the union-of-risks formula: risk_B = 1 - product_i(1 - r_i). Routing thresholds: score >= 0.80 is high risk; 0.40–0.80 is medium; below 0.40 is low.

3.5.6 Governance, Schema Registry, and Audit Log

The governance rule is unconditional: when no drift is present, data proceeds directly to production; when any drift is detected, the batch is isolated in staging regardless of the risk score. Risk scoring informs reviewers and prioritizes the review queue but

does not authorize schema change automatically. Reviewers take one of three actions: Approve and promote (activates the candidate schema version and moves staged data to production); Reject (preserves current canonical schema and marks staged data as rejected); or Rollback (reactivates a previously active schema version identified by content hash).

The schema registry stores versioned schema snapshots identified by content hash, ensuring every schema state is uniquely addressable and reproducible. The audit log records every material event with full context: drift summary, rename mappings and confidence scores, risk score and contributing factors, routing decision, and reviewer identity, timestamp, and stated reason. The log is append-only, preserving the complete history of how schema state evolved over time and satisfying regulatory audit requirements that demand traceable decision accountability.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the evaluation results for each of the four framework components. Results are drawn from the experimental evaluations conducted as described in the methodology, and are discussed in relation to the theoretical properties of each component design and the state-of-the-art benchmarks reviewed in Chapter 2.

4.2 Component 1: Unstructured Data Preprocessing Results

4.2.1 Text Processing Performance

Across a representative corpus of web-sourced text, the agentic pipeline achieved a cleaning success rate of 88–92%, with remaining records classified as partial due to single-agent failures rather than full pipeline failures. The mean overall confidence score across successfully cleaned records was 0.84 (standard deviation 0.11), reflecting consistent performance across varied input types. The LLM planner failed to produce a valid plan in approximately 2–5% of cases due to Ollama inference timeouts; in all such cases the fallback plan activated without user-visible impact.

Agent utilization rates varied substantially by input type, which is precisely the behavior the framework is designed to produce. The lang_detect_agent and tokenizer_agent executed on over 95% of records, reflecting their near-universal applicability. The html_strip_agent activated on approximately 35–45% of records, consistent with the proportion of web-scraped inputs carrying markup. The spell_agent was invoked on 40–60% of records but skipped execution on a further 15–20% of those due to the noise ratio falling below the 0.10 threshold. The ner_agent executed on 70–85% of English-language records. The translator_agent was dynamically inserted into 20–30% of plans at runtime, reflecting the multilingual composition of the test corpus.

The latency advantage of selective execution over a fixed full-pipeline approach was measured on three representative input categories. For English HTML text with spelling errors, the agentic plan selected four agents versus the full eight, achieving approximately 50% latency reduction. For clean German text requiring only language

detection, translation, and tokenization, the three-agent plan was approximately 62% faster than the full pipeline. For clean English text where spell correction was suppressed by the noise threshold, the effective plan collapsed to a single agent, yielding an 87% latency reduction. Averaged across the test corpus, selective execution produced a 35–40% mean latency reduction relative to a fixed pipeline applying all agents unconditionally.

4.2.2 Image Processing Performance

The post-transformation validation accepted 92% of agent outputs across all tested images, indicating that the decision engine's threshold-based selections were well-calibrated to the quality characteristics of the test set. The 8% of reverted transformations arose primarily from enhancement passes on images whose brightness and contrast were already close to acceptable boundaries, where CLAHE occasionally pushed SSIM below the 0.995 threshold by introducing localized over-enhancement in uniform regions.

The SSIM of accepted transformations ranged from 0.994 to 0.998, with the ROI agent consistently producing the highest SSIM scores (mean 0.997) since cropping introduces no pixel-level distortion within the retained region. The Denoise agent produced mean SSIM of 0.996, reflecting strong structural preservation by NLMeans. The Deblur agent produced the widest SSIM range (0.993 to 0.998), as performance is sensitive to the assumed PSF size relative to the actual blur kernel. The Enhancement agent produced mean PSNR of 25.3 dB, comfortably above the 15.0 dB acceptance threshold.

A direct comparison between quality-aware and blind (fixed-sequence) image processing was conducted on a set of document scans with known ground truth labels. The blind approach, applying all four agents to every image, achieved a validation acceptance rate of 68%, with 32% of transformations degrading SSIM below the threshold. The quality-aware approach achieved a 92% acceptance rate—a 24 percentage point improvement, demonstrating that agent selection based on measured deficiency is substantially more effective than unconditional processing.

4.2.3 System-Level Performance

End-to-end latency, measured from the moment of the /ingest HTTP request to the moment the processed record was available in MongoDB, ranged from 0.6 to 2.5 seconds for text records and from 2.2 to 5.4 seconds for image records. The dominant latency component for text was the LLM planning call (400–1200 ms depending on model load), while for images it was agent execution time, particularly NLMeans denoising (800–1800 ms) and Wiener deconvolution (600–1400 ms). Kafka routing latency contributed a consistent overhead of 20–100 ms per message—negligible relative to processing time but providing the architectural benefit of fully asynchronous ingestion.

Throughput was measured as sustained records processed per second under continuous load. The text pipeline achieved 6–8 records per second, limited primarily by LLM inference capacity. The image pipeline achieved 0.8–1.5 records per second, limited by the per-pixel cost of NLMeans and Wiener deconvolution. Both pipelines exhibited linear throughput scaling with the number of Kafka consumer workers up to the point where the LLM (text) or CPU (image) became the bottleneck.

4.3 Component 2: Structured Data Cleaning Results

4.3.1 Evaluation Framework

The framework is evaluated across six complementary dimensions: Rule Extraction Fidelity (precision and recall of extracted validation constraints against ground-truth domain rule annotations); Issue Detection Accuracy (true positive rate, false positive rate, and F1-score for each issue category across synthetically corrupted benchmark datasets); Correction Accuracy (MAE for numerical attributes, classification accuracy for categorical attributes, comparing model-imputed values against original uncorrupted values); Dependency Precision (proportion of dependency graph edges corresponding to genuine causal or associative relationships); HITL Review Efficiency (proportion of issues resolved through human review versus auto-fallback); and Throughput and Latency (end-to-end records per second and per-batch processing latency).

4.3.2 Dependency Architecture Validation

The central technical claim of the framework—that correcting a column using only its statistically validated predictors produces better corrections than correcting using all available columns—rests on two mechanisms. First, including unrelated columns as predictors introduces spurious correlations that the Random Forest may overfit to, producing corrections that are statistically plausible in training data but semantically incorrect in the target domain. Second, for datasets with many columns relative to rows, unrestricted feature sets create a dimensionality problem that degrades model generalization. The dependency graph provides principled dimensionality reduction preserving genuine predictive structure.

Table 4.1: Per-Agent Image Quality Metrics

Agent	Mean SSIM	Mean PSNR (dB)	Accept Rate
ROI	0.997	31.4	98%
Denoise	0.996	28.7	95%
Deblur	0.996	24.5	89%
Enhancement	0.994	25.3	91%

Table 4.2: System-Level Performance Summary

Metric	Target	Observed	Status
Text cleaning success	85%+	88-92%	Met
Mean text confidence	0.80+	0.84	Met
Image accept rate	90%+	92%	Met
Text latency	Under 3s	0.6-2.5s	Met
Image latency	Under 10s	2.2-5.4s	Met
Text throughput	5-10 rec/s	6-8 rec/s	Met
Image throughput	0.5-2 rec/s	0.8-1.5 rec/s	Met
Overall error rate	Under 5%	2-3%	Well below

The Global RF baseline (a single Random Forest trained on all non-target columns without dependency filtering) isolates the quality contribution of dependency-aware feature selection by holding the model type constant. Expected results, grounded in the theoretical properties of dependency-informed feature selection, show that the proposed framework outperforms Global RF for datasets with moderate to high inter-

column correlation, where undirected feature contamination degrades prediction quality most severely.

4.3.3 HITL Governance Effectiveness

The three-mode HITL governance structure produces a traceable record of which corrections were reviewer-approved, which were auto-filled for unreviewed issues, and which were applied through the inactivity-triggered fallback. The approval modal design reflects a deliberate asymmetry between the effort required to approve and to rollback: both actions require a single click, but rollback suppresses further auto-triggering, ensuring that a user who dismisses the modal is not immediately re-prompted. This design respects reviewer agency while maintaining the property that auto-corrections are never applied without affirmative human confirmation.

The combination of SHAP and LIME explanations serves a functional rather than decorative role: it determines whether the HITL interface enables genuine human oversight or merely its appearance. A reviewer who sees that a salary imputation is driven primarily by department and seniority can confirm or reject the suggestion in seconds, whereas without explanations, verifying the suggestion from domain knowledge alone is impractical at streaming batch scale.

4.3.4 Scalability Considerations

Dependency computation scales quadratically with the number of non-identifier columns. For wide datasets (100+ columns), this computation becomes the dominant initialization latency. Practical deployments in wide-schema environments may benefit from pre-specified dependency graphs or dimensionality reduction prior to dependency analysis. Model training scales with training set size and predictor count; for high-throughput streaming sessions, periodic rather than per-batch model retraining reduces cumulative computation cost. The Kafka-based decoupling provides architectural headroom enabling additional partitions and consumer instances without changes to processing logic.

4.4 Component 3: Privacy Preservation Results

4.4.1 Detection Accuracy

The detection pipeline was evaluated on a hybrid dataset comprising publicly available PII benchmark datasets from Kaggle and HuggingFace, augmented with synthetic PII data generated under realistic streaming conditions using Apache Airflow and Kafka schedulers. The NER model achieved a weighted F1-score of 0.913 across all supported entity types. Credit card (Luhn-validated) and NIC detection achieved 100% precision through deterministic regex validation.

Consistent with findings by Agarwal et al. [40], NER-only detection exhibited reduced recall for ambiguous entity contexts (e.g., organization names resembling common nouns). The hybrid ensemble strategy recovered approximately 11% of these missed entities through regex supplementation, yielding an overall system recall improvement of 0.08. This confirms that the dual-method architecture provides complementary coverage that neither method alone achieves.

4.4.2 Latency and Throughput

Inline privacy transformation introduced mean latency of 38 ms per document for AES-256-GCM encryption and 12 ms for masking operations at document sizes up to 50 KB. Tokenization vault lookups added an average of 6 ms per entity under in-memory storage. These figures confirm that the system satisfies sub-100 ms latency targets for high-throughput ingestion environments. Throughput testing on synthetic streaming datasets demonstrated stable processing at 1,200 documents per minute on a standard 8-core development machine.

4.4.3 Compliance Effectiveness

The leakage score—defined as the fraction of original PII entities detectable in protected output—was 0.00 for AES-256-GCM, tokenization, and bcrypt outputs, confirming zero residual machine-readable PII after strong protection. SHA-256 hashed outputs also yielded a leakage score of 0.00 against the original plaintext; the implemented salt mechanism addresses the theoretical preimage attack vulnerability. Masking preserves format structure but achieves leakage scores above 0.0 for partially visible entities by design, to retain referential utility.

All tested scenarios demonstrated compliance with GDPR Article 5 data minimization requirements, PCI-DSS v4.0 cardholder data protection mandates, and PDPA No.

9/2022 obligations for Sri Lankan personal data. The RAG-generated policy rules for GDPR and HIPAA entities matched expert-authored rules in 89% of cases by semantic similarity scoring.

4.4.4 Comparison with Existing Tools

The proposed framework was compared against five leading privacy tools. Protecto AI stores raw PII in an encrypted vault; the proposed framework masks before storage eliminating the vault exposure model entirely. Apache Airflow requires custom masking tasks and applies no native ingestion-layer masking; the proposed framework has built-in ingestion-layer masking. Dagster is developer-dependent for masking implementation; the proposed framework automates detection and masking. DataGrail focuses on lifecycle governance assuming stored PII; the proposed framework prevents raw PII storage. OneTrust masks on request only; the proposed framework enforces real-time ingestion masking. The proposed framework is the only solution that eliminates raw PII at the ingestion layer before persistence, addressing the fundamental gap identified across all reviewed commercial tools.

4.5 Component 4: Schema Drift Detection Results

4.5.1 Rename Detection Performance

The full model was evaluated against three progressively stronger baselines on 633 held-out test pairs from 40 unseen scenarios. At the selected threshold (0.5359), the full model achieved 0.80 precision, 0.90 recall, 0.85 F1, and 0.9226 PR-AUC with overall accuracy of 0.8957.

The name-similarity-only baseline achieved F1 of 0.67 and PR-AUC below 0.81, confirming that rename resolution cannot be reduced to string comparison. Adding semantic embedding similarity produced the single largest recall jump, from 0.64 to 0.85, capturing synonym substitutions and vocabulary changes that lexical features miss. Adding statistical features improved precision from 0.79 to 0.80, reducing false positive acceptances where columns appeared similar in name and semantics but differed in distributional behavior.

4.5.2 Ablation Analysis

The stepwise ablation isolates each evidence group's contribution. Lexical features alone achieved good precision (0.71) but poor recall (0.64), because many realistic renames are lexically dissimilar. Adding semantic similarity produced the largest single recall improvement, from 0.64 to 0.85, reflecting that synonym substitutions and semantic vocabulary shifts are the most common failure mode for string-based matching. Adding statistical features improved precision by filtering pairs that appeared similar but differed in distributional behavior.

The 18-point F1 gap over the name-similarity baseline reflects the breadth of rename patterns in real pipelines. Failure cases cluster around abbreviation with no shared tokens, synonym substitution, and token reordering—all requiring semantic evidence. The scenario-level train-test split requires genuine generalization to unseen schema structures rather than interpolation within known scenarios. Achieving 0.85 F1 under this stricter protocol is a more meaningful signal of operational readiness than pair-level numbers would be.

4.5.3 Governance Design Effectiveness

Table 4.3: Protection Method Selection by Risk Score

Method	Applicable Entity Types	Risk Score	Description
MASKING	EMAIL, PHONE, DATE	≤2.0	Partial obscuration preserving format
SHA-256 HASH	PER, EMAIL, LOC	2.0–3.5	One-way salted hash; non-reversible
AES-256-GCM	NIC, DATE, BANK_ACCOUNT	3.5–4.5	Authenticated symmetric encryption; reversible with key
TOKENIZATION	CREDIT_CARD, BANK_ACCT	>4.5	Vault-based replacement; detokenizable
BCRYPT	PASSWORD	Always	Adaptive-cost password hashing
GENERALIZATION	DATE, LOC	Low	Date → year only; City → Country

The unconditional staging gate reflects a deliberate trade-off. Auto-approving low-risk events reduces reviewer workload but silently accepts schema changes on the assumption that the risk model is always correct. For regulated datasets, the audit trail of who approved what and when is the mechanism through which accountability is

established when downstream failures occur. The three-action governance design (approve, reject, rollback) recognizes that governance over ambiguous structural change requires more than a binary yes/no decision—the ability to revert a previous acceptance is essential for maintaining schema integrity over time.

The components are mutually reinforcing within the framework: normalization reduces false candidates before rename resolution; rename resolution corrects the diff before risk scoring; risk scoring with explanations gives reviewers the context to make sound decisions. Removing any one stage degrades the others. The staging gate, the approve/reject/rollback workflow, and the audit log are not overhead—they are the mechanism through which detection becomes operationally useful and accountable.

4.6 Cross-Component Discussion

Considered together, the four components demonstrate a consistent design philosophy across distinct technical domains: adaptive decision-making based on input characteristics, human accountability as the primary pathway with automation as a safely constrained fallback, complete auditability of every pipeline decision, and modular architecture enabling independent deployment or integrated operation.

The asymmetry between the LLM-guided text planner and the rule-based image decision engine in Component 1 illustrates a broader design principle: use LLMs where decisions require reading and interpreting content, and use deterministic metric-based logic where quality can be measured objectively. This principle extends naturally to the other components: Component 2 uses LLM rule extraction (where constraint derivation requires semantic understanding) but deterministic Random Forest prediction (where structured tabular correction can be learned from data); Component 3 uses transformer NER (where entity recognition requires contextual language understanding) alongside deterministic regex (where structured PII types can be identified with certainty); Component 4 uses learned semantic embeddings (where rename detection requires semantic reasoning) alongside deterministic normalization (where naming convention variation can be resolved algorithmically).

The Kafka backbone shared by Components 1 and 2, and the API-driven ingestion architecture of Components 3 and 4, reflect the principle that data quality and

governance decisions should operate as close to the data source as possible. All four components enforce their core invariants—clean preprocessing, dependency-coherent corrections, zero raw PII at rest, staged schema changes—before data reaches downstream consumers, rather than after.

Table 4.4: Rename Detection Model Comparison

Model	Precision	Recall	F1	PR-AUC
Name similarity only	0.71	0.64	0.67	—
Lexical + type	0.77	0.74	0.75	0.81
Lexical + semantic	0.79	0.85	0.82	0.89
Full model (all features)	0.80	0.90	0.85	0.92

CHAPTER 5: CONCLUSIONS AND RECOMMENDATIONS

5.1 Summary of Contributions

This dissertation has presented an Autonomous Framework for Real-Time Data Quality, Privacy, and Semantic Integrity in Streaming Environments—a four-component system that advances the state of the art across unstructured data preprocessing, structured data quality management, privacy preservation, and schema drift governance.

Component 1 demonstrated that combining LLM-guided planning for text preprocessing with quantitative quality-metric-based agent selection for image preprocessing, within a Kafka-native streaming architecture, produces a more adaptive and resource-efficient alternative to static preprocessing pipelines. The 35–40% mean latency reduction and 24 percentage point improvement in image transformation acceptance rate over blind fixed-sequence processing confirm the value of adaptive agent selection.

Component 2 established that a dependency-informed per-column Random Forest prediction engine, trained exclusively on statistically validated predictors, prevents spurious corrections from unrelated attributes and produces contextually coherent repairs. The HITL governance layer with SHAP and LIME explanations ensures that corrections are both accurate and accountable, with human review as the default and automation as a safely constrained fallback.

Component 3 confirmed that ingestion-layer PII enforcement achieves a leakage score of 0.00 for strong protection methods (AES-256-GCM, tokenization, bcrypt) at sub-100 ms per-document latency, with F1 > 0.91 for the hybrid NER-regex detection ensemble. The RAG-based policy enhancement subsystem enables continuous regulatory adaptation without code modifications.

Component 4 demonstrated that rename-aware, risk-informed drift handling supports reliable and auditable schema adaptation in production pipelines. The rename model achieved F1 = 0.85 and PR-AUC = 0.9226 under scenario-level generalization, and

the unconditional staging gate with governed human review produces a complete, traceable accountability record for every schema change decision.

5.2 Research Findings

Several cross-cutting findings emerge from this research. First, adaptive processing consistently outperforms static processing across all four components: dynamic agent selection reduces unnecessary computation, dependency-aware models reduce spurious corrections, risk-based protection calibrates method strength to entity sensitivity, and risk-informed staging prioritizes reviewer attention appropriately. Second, human accountability and automated assistance are complementary rather than competing: in all four components, the most effective design positions automated decision-making as a tool that informs and accelerates human judgment rather than replacing it. Third, auditability is not overhead but rather the mechanism through which automated data governance decisions become operationally useful in regulated environments.

The research also surfaces a consistent finding about the appropriate boundary between LLM-based and deterministic reasoning: LLMs add value where decisions require reading and interpreting content (text preprocessing planning, validation rule extraction from documents, rename detection through semantic embedding); deterministic logic is preferable where quality can be measured objectively or where correctness requires formal guarantees (image quality metrics, Luhn validation, NIC format verification, column normalization).

5.3 Limitations

Several limitations of the current implementation warrant acknowledgment. In Component 1, the Wiener deconvolution assumes a uniform box point-spread function, which holds reasonably for defocus blur but breaks down for motion blur. The LLM planner adds variable latency that is difficult to bound under production load contention, and neither planner nor rule-based engine learns from prior decisions.

In Component 2, dependency computation scales quadratically with column count, which becomes the dominant initialization latency for wide datasets. The framework

applies corrections sequentially rather than enforcing multi-constraint consistency jointly, leaving a residual limitation relative to HoloClean's holistic repair approach for dense constraint dependencies.

In Component 3, the XLM-RoBERTa model requires GPU availability for production-scale throughput. The regex engine is calibrated for Sri Lankan structured PII formats and would require extension for other jurisdictional identifiers. The RAG policy enhancement is bounded by the accuracy of Google Gemini-generated policy recommendations.

In Component 4, the rename model was trained and evaluated on a 160-scenario corpus spanning five domains, which establishes proof of concept but may not fully represent the breadth of rename patterns across all enterprise schema naming conventions. The governance workflow currently supports a single review step; regulated datasets may benefit from multi-step review workflows.

5.4 Future Work

The following directions are identified for future investigation across all four components:

Component 1: Integration of learned neural deblurring models to handle directional motion blur PSFs; implementation of a timeout-based LLM fallback mechanism for production latency bounds; development of a feedback loop enabling agent plan quality to improve from downstream model performance signals; extension to audio preprocessing with SNR and clipping rate metrics.

Component 2: Empirical evaluation with human subjects measuring time-to-review per issue, rate of human deviation from ML suggestions, and quality delta of human corrections versus auto-corrections; online learning integration incorporating reviewer selections as labeled training signal; joint consistency enforcement extending the correction stage with a constraint satisfaction layer; retrieval-augmented rule extraction replacing direct LLM prompting with domain-specific corpora.

Component 3: Extension to enterprise-scale orchestration with Apache Spark; implementation of homomorphic encryption for compute-on-encrypted-data analytics;

adaptive differential privacy budgets; a federated variant processing PII locally at edge nodes without centralizing raw data.

Component 4: Larger evaluation corpus with schemas from hundreds of columns across broader domain naming conventions; retrieval-and-reranking upgrade following the ReMatch architecture; multi-step governance workflow for regulated datasets with technical impact assessment followed by compliance confirmation.

Additionally, future work will investigate the integration of all four components into a unified streaming governance platform with shared observability instrumentation, enabling cross-component data lineage tracking and end-to-end quality certification for streaming data assets.

5.5 Summary of Each Student's Contribution

IT22549136 – Amodhya Sharinda Silva

Amodhya Sharinda Silva led the design, implementation, and evaluation of Component 1: the Agentic Multi-Modal Unstructured Data Preprocessing Framework. Contributions include: the LLM-guided agent planning architecture using Ollama and LangChain; the eight-agent text processing pipeline with dynamic plan modification; the rule-based decision engine for image agent selection based on Laplacian variance, luminance, and contrast metrics; the PSNR/SSIM post-transformation validation mechanism with automatic reversion; the five-topic Apache Kafka streaming architecture; MongoDB persistence; and the React SSE monitoring frontend. Silva authored the corresponding research paper accepted for presentation at an international IEEE-indexed conference in 2026.

IT22893666 – L.T.E. Arachchi

L.T.E. Arachchi led the design, implementation, and evaluation of Component 2: the Scalable Framework for Structured Data Cleaning. Contributions include: the dependency-informed per-column ML prediction architecture using chi-square testing, mutual information, and Spearman correlation for dependency graph construction; the Random Forest Classifier and Regressor pipeline with ColumnTransformer preprocessing; the SHAP TreeExplainer and LIME explanation integration; the three-

mode HITL governance layer with inactivity-triggered preview and mandatory approval; the LLaMA 3.3-70B LLM rule extraction via Groq API; the composite quality scoring mechanism; and the Kafka-based distributed streaming infrastructure with Zookeeper. Arachchi authored the corresponding IEEE-format research paper.

IT22556684 – H.A. Amanda K. Arangalla

H.A. Amanda K. Arangalla led the design, implementation, and evaluation of Component 3: the Autonomous Framework for Real-Time PII Detection and Privacy Preservation. Contributions include: the XLM-RoBERTa NER fine-tuning for multilingual PII detection with BIO tagging; the deterministic regex extraction engine for Sri Lankan structured PII types including NIC, credit card (Luhn-validated), bank account, and phone number formats; the priority-based entity merging and deduplication algorithm; the regulation-aware policy engine encoding GDPR, HIPAA, CCPA, PCI-DSS v4.0, and PDPA No. 9/2022; the adaptive protection module with AES-256-GCM, tokenization, SHA-256, bcrypt, and generalization methods; and the RAG subsystem using ChromaDB and Google Gemini for dynamic policy enhancement. Arangalla authored the corresponding research paper.

IT22204752 – W.R.W.M.N.S. Weerakoon

W.R.W.M.N.S. Weerakoon led the design, implementation, and evaluation of Component 4: the Framework for Schema Drift Detection and Governed Adaptation. Contributions include: the deterministic column normalization pipeline; the conservative type inference algorithm with Boolean-first precedence ordering; the structural change identification framework based on the Chillón et al. taxonomy; the eight-feature multi-source rename detection model combining lexical, semantic, structural, and statistical evidence; the trained probabilistic classifier with scenario-level generalization evaluation; the dual-mode risk scoring architecture (Option A dataset-level and Option B field-level with union-of-risks formulation); the unconditional staging gate; the approve/reject/rollback governance workflow; the content-hash-based schema registry; and the append-only audit log. Weerakoon authored the corresponding research paper.

REFERENCES

[1] IDC, "Data Age 2025: The Digitization of the World from Edge to Core," Seagate, Nov. 2018.

[2] IBM, "The Four V's of Big Data," IBM Big Data & Analytics Hub, 2021.

[3] W. Fan and F. Geerts, Foundations of Data Quality Management, Morgan & Claypool, 2012.

[4] M. Kantarcioglu and F. Shaon, "Securing big data in the age of AI," in Proc. 1st IEEE Int. Conf. Trust, Privacy Security Intell. Syst. Appl. (TPS-ISA), 2019.

[5] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," Proc. VLDB Endow., vol. 10, no. 11, pp. 1190–1201, Aug. 2017.

[6] R. K. Vaddepalli, "AutoSchema: A self-learning framework for detecting and adapting to schema drift in real-time data streams," Eur. J. Adv. Eng. Technol., vol. 10, no. 7, pp. 94–100, 2023.

[7] A. V., G. R. Gangadharan, and R. Buyya, "Agentic artificial intelligence (AI): Architectures, taxonomies, and evaluation of large language model agents," arXiv preprint arXiv:2601.12560, 2026.

[8] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. ICLR, 2023.

[9] Y. Shen et al., "HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face," arXiv preprint arXiv:2303.17580, 2023.

[10] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python, O'Reilly Media, 2009.

[11] M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd, "spaCy: Industrial-strength NLP in Python," Zenodo, 2020.

[12] M. Lui and T. Baldwin, "langid.py: An off-the-shelf language identification tool," in Proc. ACL 2012 System Demos, pp. 25–30, 2012.

[13] S. Pertuz, D. Puig, and M. A. Garcia, "Analysis of focus measure operators for shape-from-focus," Pattern Recognition, vol. 46, no. 5, pp. 1415–1432, 2013.

[14] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Trans. Image Processing, vol. 13, no. 4, pp. 600–612, Apr. 2004.

[15] K. J. Zuiderveld, "Contrast limited adaptive histogram equalization," in Graphics Gems IV, Academic Press, 1994, pp. 474–485.

[16] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in Proc. 6th Int. Conf. Computer Vision (ICCV), pp. 839–846, 1998.

[17] A. Buades, B. Coll, and J.-M. Morel, "A non-local algorithm for image denoising," in Proc. IEEE CVPR, vol. 2, pp. 60–65, 2005.

[18] J. Canny, "A computational approach to edge detection," IEEE Trans. Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679–698, 1986.

[19] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in Proc. NetDB Workshop, 2011.

[20] S. Bradshaw, E. Brazil, and K. Chodorow, MongoDB: The Definitive Guide, 3rd ed., O'Reilly Media, 2019.

[21] W. Fan and F. Geerts, "Foundations of Data Quality Management," Synthesis Lectures on Data Management, Morgan & Claypool, 2012.

[22] G. Cong, W. Fan, F. Geerts, X. Jia, and S. Ma, "Improving data quality: Consistency and accuracy," Proc. 33rd VLDB, Vienna, 2007, pp. 315–326.

[23] O. Troyanskaya et al., "Missing value estimation methods for DNA microarrays," Bioinformatics, vol. 17, no. 6, pp. 520–525, Jun. 2001.

[24] J. Yoon, J. Jordon, and M. van der Schaar, "GAIN: Missing data imputation using generative adversarial nets," in Proc. ICML, 2018, pp. 5689–5698.

[25] T. Rekatsinas et al., "HoloClean: Holistic data repairs with probabilistic inference," Proc. VLDB Endow., vol. 10, no. 11, pp. 1190–1201, Aug. 2017.

[26] I. F. Ilyas and X. Chu, Data Cleaning, Morgan & Claypool, 2019.

[27] S. Krishnan et al., "ActiveClean: Interactive data cleaning for statistical modeling," Proc. VLDB Endow., vol. 9, no. 12, pp. 948–959, 2016.

[28] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems, 2017, pp. 4765–4774.

[29] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. ACM SIGKDD, 2016, pp. 1135–1144.

[30] A. Bontempelli et al., "Human-in-the-loop handling of knowledge drift," Data Min. Knowl. Discov., 2022.

[31] Y. Li et al., "Table-GPT: Table-tuned GPT for diverse table tasks," arXiv preprint arXiv:2310.09263, Oct. 2023.

[32] P. Peeters, R. Balayn, and C. Lofi, "Entity matching using large language models," arXiv preprint arXiv:2310.11244, Oct. 2023.

[33] G. R. Mittoor, P. Udayaraju, and S. Putteti, "Building privacy-aware data pipelines: Balancing scalability, compliance, and performance," in Proc. ICMLAS-2025, IEEE, 2025.

[34] P. Mahendra and A. Verma, "Privacy-preserving data pipelines for AI: A comprehensive review of scalable approaches," in Proc. ICICI-2025, IEEE, 2025.

[35] C. Dwork, "Differential privacy: A survey of results," in Theory and Applications of Models of Computation, Springer, 2008, pp. 1–19.

[36] M. Kantarcioglu and F. Shaon, "Securing big data in the age of AI," in Proc. TPS-ISA, IEEE, 2019.

[37] NIST, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)," NIST SP 800-38D, Nov. 2007.

[38] H. Kotadia et al., "Named entity recognition in unstructured medical text documents," arXiv preprint arXiv:2110.15732, 2021.

[39] Anon., "A hybrid rule-based NLP and machine learning approach for PII detection in financial documents," Sci. Rep., 2025, doi: 10.1038/s41598-025-04971-9.

[40] P. Agarwal et al., "Unmasking the reality of PII masking models: Performance gaps and the call for accountability," arXiv preprint arXiv:2504.12308, 2025.

[41] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," in Proc. ACL, 2020, pp. 8440–8451.

[42] Microsoft, "Presidio — open-source framework for PII detection and anonymization," GitHub, 2024. [Online]. Available: https://github.com/microsoft/presidio

[43] T. P. Raptis and A. Passarella, "A survey on networked data streaming with Apache Kafka," IEEE Access, vol. 11, pp. 85333–85350, 2023.

[44] Y. Zhao et al., "PrivStream: A privacy-preserving inference framework on IoT streaming data at the edge," Inf. Fusion, vol. 80, pp. 163–177, 2022.

[45] P. Kodakandla, "Architecting privacy-centric data pipelines with generative AI," Int. J. Sci. Res. Arch., vol. 13, no. 2, pp. 1502–1512, 2024.

[46] Y.-H. Chen, H.-H. Chen, and P.-C. Huang, "Enhancing the data privacy for public data lakes," in Proc. IEEE ICASI, 2018, pp. 1065–1068.

[47] European Parliament, "Regulation (EU) 2016/679 (General Data Protection Regulation)," Official Journal of the European Union, L 119, pp. 1–88, May 2016.

[48] U.S. Department of Health and Human Services, "Health Insurance Portability and Accountability Act of 1996 (HIPAA)," Public Law 104-191, Aug. 1996.

[49] California State Legislature, "California Consumer Privacy Act (CCPA)," Assembly Bill 375, Jun. 2018.

[50] Parliament of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," Gazette Extraordinary No. 2267/40, Mar. 2022.

[51] P. Chillón, M. Klettke, D. S. Ruiz, and J. G. Molina, "A generic schema evolution approach for NoSQL and relational databases," IEEE Trans. Knowl. Data Eng., 2024.

[52] A. H. Chillón, M. Klettke, D. S. Ruiz, and J. G. Molina, "Schema drift: Relational concept stability across repeated comparison," unpublished.

[53] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," VLDB J., vol. 10, no. 4, pp. 334–350, 2001.

[54] S. K. Chintakindhi, "AI-driven schema drift detection: Automating regulatory compliance in cloud migration projects," IJIRMPS, vol. 13, no. 2, Mar.–Apr. 2025.

[55] R. K. Vaddepalli, "AutoSchema: A self-learning framework for detecting and adapting to schema drift in real-time data streams," Eur. J. Adv. Eng. Technol., vol. 10, no. 7, pp. 94–100, 2023.

[56] A. Sirimalla, "Automated schema drift detection and resolution in multi-database CI/CD pipelines," Frontiers Comput. Sci. Artif. Intell., 2024.

[57] A. Chitnis and S. Tewari, "Detecting data drift and ensuring observability with machine learning automation," JETIR, vol. 9, no. 8, Aug. 2022.

[58] M. Asif-Ur-Rahman et al., "A semi-automated hybrid schema matching framework for vegetation data integration," arXiv, 2023. https://arxiv.org/abs/2305.06528

[59] D. U. Priya and P. S. Thilagam, "JSON document clustering based on schema embeddings," J. Inf. Sci., Sep. 2022.

[60] E. Sheetrit, M. Brief, M. Mishaeli, and O. Elisha, "ReMatch: Retrieval enhanced schema matching with LLMs," arXiv, Mar. 2024.

[61] Y. Lee et al., "Explainable artificial intelligence-based model drift detection applicable to unsupervised environments," CMC, vol. 76, no. 2, 2023.

[62] R. Abraham, J. Schneider, and J. vom Brocke, "Data governance: A conceptual framework, structured review, and research agenda," Int. J. Inf. Manag., vol. 49, pp. 424–438, Dec. 2019.

[63] T. Kiss and J. Strunk, "Unsupervised multilingual sentence boundary detection," Computational Linguistics, vol. 32, no. 4, pp. 485–525, 2006.

[64] ITU-R, Recommendation ITU-R BT.601-7: Studio Encoding Parameters of Digital Television, 2011.

[65] F. Orieux, J.-F. Giovannelli, and T. Rodet, "Bayesian estimation of regularization and point spread function parameters for Wiener-Hunt deconvolution," J. Optical Society of America A, vol. 27, no. 7, pp. 1593–1607, 2010.

[66] S. Ramírez, FastAPI Documentation, 2018. [Online]. Available: https://fastapi.tiangolo.com

[67] Pydantic, Pydantic Documentation: Data Validation Using Python Type Hints. [Online]. Available: https://docs.pydantic.dev

[68] Google Cloud, Cloud Translation API Documentation. [Online]. Available: https://cloud.google.com/translate/docs

[69] M. Alkaeed, "Privacy preservation in artificial intelligence and extended reality systems," J. Netw. Comput. Appl., vol. 237, Art. no. 103215, 2024.

[70] C. Gilbert and M. A. Gilbert, "Privacy-preserving data mining and analytics in big data environments," SSRN Electron. J., 2024.

[71] H. Kumar and M. A. Ismail, "Big data streaming platforms: A review," Iraqi J. Comput. Sci. Math., vol. 3, no. 2, pp. 95–100, 2022.

[72] X. Zhu, J. Wu, and S. Chen, "Real-time data quality monitoring for streaming data," in Proc. IEEE Int. Conf. Big Data, Los Angeles, 2019, pp. 2812–2821.

[73] A. Bifet, G. Holmes, and B. Pfahringer, "MOA: Massive online analysis," J. Mach. Learn. Res., vol. 11, pp. 1601–1604, Aug. 2010.